

Logistic Regression and Gradient Descent

CSE427: Machine Learning

Learning Objectives

By the end of this lecture, students should be able to:

- 1 Explain why logistic regression is needed for classification.
- 2 Connect linear classifiers with logistic regression.
- 3 Explain sigmoid, log-odds, and probability prediction.
- 4 Compute logistic regression prediction by hand.
- 5 Explain binary cross-entropy loss.
- 6 Explain gradient descent intuition.
- 7 Perform one full gradient descent update.

Motivation: What Problem Are We Solving?

Suppose we want to predict whether a student passes an exam.

$$y = \begin{cases} 1, & \text{Pass} \\ 0, & \text{Fail} \end{cases}$$

Possible features:

x_1 = hours studied, x_2 = number of practice tests

Goal

Given x_1 and x_2 , predict whether the student will pass or fail.

But we also want a probability:

$$P(y = 1 \mid x)$$

Why Not Linear Regression?

Linear regression predicts:

$$\hat{y} = w_1x_1 + w_2x_2 + b$$

But classification needs:

$$0 \leq P(y = 1 \mid x) \leq 1$$

Problem

Linear regression can output values like:

$$-1.5, \quad 2.3, \quad 7.8$$

These are not valid probabilities.

So we need a model that keeps the output between 0 and 1.

Linear Classifier Idea

A linear classifier first computes a score:

$$z = w_1x_1 + w_2x_2 + b$$

or generally:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

Decision rule:

$$z > 0 \Rightarrow y = 1$$

$$z \leq 0 \Rightarrow y = 0$$

Idea

The score z tells which side of the decision boundary the data point belongs to.

What Logistic Regression Adds

A linear classifier gives a hard decision.

Logistic regression gives a probability.

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

$$\hat{y} = P(y = 1 \mid \mathbf{x}) = \sigma(z)$$

where $\sigma(z)$ is the sigmoid function.

Main Idea

Logistic regression uses a linear score, then converts it into a probability.

Sigmoid Function

The sigmoid function is:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

It maps any real number into a probability:

$$-\infty < z < +\infty$$

$$0 < \sigma(z) < 1$$

z	$\sigma(z)$	Meaning
Large negative	Close to 0	Class 0 likely
0	0.5	Boundary
Large positive	Close to 1	Class 1 likely

Important Sigmoid Values

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

z	Calculation	$\sigma(z)$
-2	$\frac{1}{1+e^2}$	0.119
-1	$\frac{1}{1+e^1}$	0.269
0	$\frac{1}{1+e^0}$	0.500
1	$\frac{1}{1+e^{-1}}$	0.731
2	$\frac{1}{1+e^{-2}}$	0.881

$$z > 0 \Rightarrow \sigma(z) > 0.5, \quad z < 0 \Rightarrow \sigma(z) < 0.5$$

Prediction Rule

Logistic regression predicts:

$$\hat{y} = P(y = 1 \mid x) = \sigma(\mathbf{w} \cdot \mathbf{x} + b)$$

Then:

$$P(y = 0 \mid x) = 1 - \hat{y}$$

Common decision rule:

$$\hat{y} \geq 0.5 \Rightarrow \text{Predict 1}$$

$$\hat{y} < 0.5 \Rightarrow \text{Predict 0}$$

The decision boundary is:

$$\mathbf{w} \cdot \mathbf{x} + b = 0$$

Where Does Logistic Regression Come From?

Let:

$$p = P(y = 1 \mid \mathbf{x})$$

Odds:

$$\frac{p}{1-p}$$

Log-odds:

$$\log \left(\frac{p}{1-p} \right)$$

Logistic regression assumes:

$$\log \left(\frac{p}{1-p} \right) = \mathbf{w} \cdot \mathbf{x} + b$$

From Log-Odds to Sigmoid

Start with:

$$z = \log \left(\frac{p}{1-p} \right)$$

Then:

$$e^z = \frac{p}{1-p}$$

$$e^z(1-p) = p$$

$$e^z - e^z p = p$$

$$e^z = p(1 + e^z)$$

$$p = \frac{e^z}{1 + e^z}$$

Meaning of Weights

For:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

- $w_j > 0$: increasing x_j increases probability of class 1.
- $w_j < 0$: increasing x_j decreases probability of class 1.
- Large $|w_j|$: feature has stronger influence.
- b : bias or intercept.

Example

If x_1 is study hours and $w_1 > 0$, more study hours increase the probability of passing.

Solved Example 1: Prediction

A student has:

$$x_1 = 3 \quad \text{hours studied}$$

$$x_2 = 2 \quad \text{practice tests}$$

Given:

$$w_1 = 0.8, \quad w_2 = 0.5, \quad b = -2$$

Compute:

$$z = w_1 x_1 + w_2 x_2 + b$$

$$z = (0.8)(3) + (0.5)(2) - 2$$

$$z = 2.4 + 1.0 - 2 = 1.4$$

Solved Example 1: Probability

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Since:

$$z = 1.4$$

$$\hat{y} = \frac{1}{1 + e^{-1.4}}$$

$$e^{-1.4} \approx 0.2466$$

$$\hat{y} = \frac{1}{1 + 0.2466}$$

$$\hat{y} = 0.802$$

Why Do We Need Training?

In the previous example, the weights were already given:

$$w_1 = 0.8, \quad w_2 = 0.5, \quad b = -2$$

But in real machine learning, we do not know them.

Training Goal

Learn $w_1, w_2, \dots, w_n, b$ from data.

To train logistic regression, we need:

- 1 A loss function.
- 2 An optimization method.

Loss: Binary Cross-Entropy

Optimizer: Gradient Descent

Loss Function Intuition

A loss function measures how wrong the prediction is.

If:

$$y = 1$$

we want:

$$\hat{y} \approx 1$$

If:

$$y = 0$$

we want:

$$\hat{y} \approx 0$$

Good Prediction

Small loss

Binary Cross-Entropy Loss

For one example:

$$L(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

If $y = 1$:

$$L = -\log(\hat{y})$$

If $y = 0$:

$$L = -\log(1 - \hat{y})$$

Meaning

The loss becomes small when the model gives high probability to the correct class.

Solved Loss Example

Case 1:

$$y = 1, \quad \hat{y} = 0.80$$

$$L = -\log(0.80) = 0.223$$

$$L = 0.223$$

Case 2:

$$y = 0, \quad \hat{y} = 0.80$$

$$L = -\log(1 - 0.80)$$

$$L = -\log(0.20) = 1.609$$

Loss Over All Training Examples

For m training examples:

$$J(\mathbf{w}, b) = \frac{1}{m} \sum_{i=1}^m L(\hat{y}^{(i)}, y^{(i)})$$

where:

$$\hat{y}^{(i)} = \sigma(\mathbf{w} \cdot \mathbf{x}^{(i)} + b)$$

Training goal:

$$\min_{\mathbf{w}, b} J(\mathbf{w}, b)$$

Meaning

Choose weights and bias that minimize average loss.

Gradient Descent Intuition

Gradient descent is an algorithm for minimizing loss.

Hill Analogy

Imagine standing on a hill in fog. You feel the slope and take a small step downhill. Repeat until you reach a low point.

Generic update rule:

$$\theta := \theta - \eta \frac{\partial J}{\partial \theta}$$

where:

- θ is a parameter.
- η is the learning rate.
- Gradient tells the direction of steepest increase.

We subtract the gradient to move toward lower loss.

Learning Rate

The learning rate is:

$$\eta$$

It controls step size.

Learning Rate	Effect
Too small	Training is very slow
Too large	May overshoot the minimum
Reasonable	Loss decreases smoothly

Classroom Line

Gradient tells direction. Learning rate tells how far to move.

Simple Gradient Descent Example

Suppose:

$$J(w) = (w - 3)^2$$

Minimum occurs at:

$$w = 3$$

Derivative:

$$\frac{dJ}{dw} = 2(w - 3)$$

Update rule:

$$w := w - \eta 2(w - 3)$$

Let:

$$w = 0, \quad \eta = 0.1$$

Simple Gradient Descent Example Continued

Iteration 2:

$$w = 0.6$$

$$\frac{dJ}{dw} = 2(0.6 - 3) = -4.8$$

$$w_{new} = 0.6 - 0.1(-4.8) = 1.08$$

Iteration 3:

$$w = 1.08$$

$$\frac{dJ}{dw} = 2(1.08 - 3) = -3.84$$

$$w_{new} = 1.08 - 0.1(-3.84) = 1.464$$

Gradient for Logistic Regression

For one example:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

$$\hat{y} = \sigma(z)$$

Binary cross-entropy loss:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

The gradients become:

$$\frac{\partial L}{\partial w_j} = (\hat{y} - y)x_j$$

$$\frac{\partial L}{\partial b} = \hat{y} - y$$

Important

Logistic Regression Update Rules

For one example:

$$w_j := w_j - \eta(\hat{y} - y)x_j$$

$$b := b - \eta(\hat{y} - y)$$

For two features:

$$w_1 := w_1 - \eta(\hat{y} - y)x_1$$

$$w_2 := w_2 - \eta(\hat{y} - y)x_2$$

$$b := b - \eta(\hat{y} - y)$$

Interpretation

If prediction is too low for class 1, weights increase. If prediction is too high, weights decrease.

Solved Example 2: One Gradient Update

Suppose one training example:

$$x_1 = 3, \quad x_2 = 2$$

True label:

$$y = 1$$

Initial parameters:

$$w_1 = 0, \quad w_2 = 0, \quad b = 0$$

Learning rate:

$$\eta = 0.1$$

We perform one gradient descent update.

Step 1 and 2: Compute z and $\hat{y}$

Compute linear score:

$$z = w_1x_1 + w_2x_2 + b$$

$$z = (0)(3) + (0)(2) + 0 = 0$$

$$\boxed{z = 0}$$

Compute prediction:

$$\hat{y} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\hat{y} = \frac{1}{1 + e^0} = \frac{1}{2} = 0.5$$

$$\boxed{\hat{y} = 0.5}$$

Step 3: Compute Loss

True label:

$$y = 1$$

Prediction:

$$\hat{y} = 0.5$$

Binary cross-entropy:

$$L = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

Since $y = 1$:

$$L = -\log(\hat{y})$$

$$L = -\log(0.5)$$

Step 4 and 5: Error and Gradients

Error term:

$$\hat{y} - y = 0.5 - 1 = -0.5$$

Gradients:

$$\frac{\partial L}{\partial w_1} = (\hat{y} - y)x_1 = (-0.5)(3) = -1.5$$

$$\frac{\partial L}{\partial w_2} = (\hat{y} - y)x_2 = (-0.5)(2) = -1.0$$

$$\frac{\partial L}{\partial b} = \hat{y} - y = -0.5$$

$$\nabla L = \begin{bmatrix} -1.5 \\ -1.0 \\ -0.5 \end{bmatrix}$$

Step 6: Update Parameters

Update rule:

$$w_j^{new} = w_j - \eta \frac{\partial L}{\partial w_j}$$

$$b^{new} = b - \eta \frac{\partial L}{\partial b}$$

Given:

$$\eta = 0.1$$

$$w_1^{new} = 0 - 0.1(-1.5) = 0.15$$

$$w_2^{new} = 0 - 0.1(-1.0) = 0.10$$

$$b^{new} = 0 - 0.1(-0.5) = 0.05$$

Step 7: Check New Prediction

Use updated parameters:

$$w_1 = 0.15, \quad w_2 = 0.10, \quad b = 0.05$$

$$x_1 = 3, \quad x_2 = 2$$

$$z = (0.15)(3) + (0.10)(2) + 0.05$$

$$z = 0.45 + 0.20 + 0.05 = 0.70$$

$$\hat{y} = \sigma(0.70) = \frac{1}{1 + e^{-0.70}}$$

$$\hat{y} = 0.668$$

$\hat{y} = 0.668$

Did the Update Help?

Before update:

$$\hat{y} = 0.5$$

After update:

$$\hat{y} = 0.668$$

True label:

$$y = 1$$

So the probability of the correct class increased.

Old loss:

$$-\log(0.5) = 0.693$$

New loss:

Second Case: If True Label is 0

Use the same input:

$$x_1 = 3, \quad x_2 = 2$$

but:

$$y = 0$$

Initial:

$$w_1 = 0, \quad w_2 = 0, \quad b = 0$$

Then:

$$z = 0, \quad \hat{y} = 0.5$$

Error:

$$\hat{y} - y = 0.5 - 0 = 0.5$$

Second Case: Update

With:

$$\eta = 0.1$$

$$w_1^{new} = 0 - 0.1(1.5) = -0.15$$

$$w_2^{new} = 0 - 0.1(1.0) = -0.10$$

$$b^{new} = 0 - 0.1(0.5) = -0.05$$

$$w_1 = -0.15, \quad w_2 = -0.10, \quad b = -0.05$$

Now:

$$z = (-0.15)(3) + (-0.10)(2) - 0.05$$

Batch Gradient Descent

For all m training examples:

$$\frac{\partial J}{\partial w_j} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)}) x_j^{(i)}$$

$$\frac{\partial J}{\partial b} = \frac{1}{m} \sum_{i=1}^m (\hat{y}^{(i)} - y^{(i)})$$

Update:

$$w_j := w_j - \eta \frac{\partial J}{\partial w_j}$$

$$b := b - \eta \frac{\partial J}{\partial b}$$

Meaning

Batch gradient descent uses all training examples before one update.

Mini Batch Example: Data

Suppose two examples:

$$(x_1, x_2, y)$$

$$(3, 2, 1), \quad (1, 1, 0)$$

Initial parameters:

$$w_1 = 0, \quad w_2 = 0, \quad b = 0$$

Learning rate:

$$\eta = 0.1$$

For both examples:

$$z = 0$$

Mini Batch Example: Gradients

For w_1 :

$$\begin{aligned}\frac{\partial J}{\partial w_1} &= \frac{1}{2}[(-0.5)(3) + (0.5)(1)] \\ &= \frac{1}{2}[-1.5 + 0.5] = -0.5\end{aligned}$$

For w_2 :

$$\begin{aligned}\frac{\partial J}{\partial w_2} &= \frac{1}{2}[(-0.5)(2) + (0.5)(1)] \\ &= \frac{1}{2}[-1 + 0.5] = -0.25\end{aligned}$$

For b :

$$\frac{\partial J}{\partial b} = \frac{1}{2}[-0.5 + 0.5] = 0$$

Mini Batch Example: Update

Update:

$$w_1^{new} = 0 - 0.1(-0.5) = 0.05$$

$$w_2^{new} = 0 - 0.1(-0.25) = 0.025$$

$$b^{new} = 0 - 0.1(0) = 0$$

$$w_1 = 0.05, \quad w_2 = 0.025, \quad b = 0$$

Observation

The positive example has larger feature values, so the weights become positive.

Feature Scaling

Gradient descent works better when features have similar scales.

Example:

x_1 = hours studied, range 0 to 10

x_2 = income, range 10000 to 100000

Large-scale features can dominate updates.

Standardization:

$$x'_j = \frac{x_j - \mu_j}{\sigma_j}$$

Min-max normalization:

$$x'_j = \frac{x_j - \min(x_j)}{\max(x_j) - \min(x_j)}$$

Types of Gradient Descent

Type	Uses	Comment
Batch GD	All examples	Stable but slow
Stochastic GD	One example	Fast but noisy
Mini-batch GD	Small batch	Common in practice

Practical Note

Mini-batch gradient descent balances speed and stability.

Training Algorithm

① Initialize weights and bias.

② For each epoch:

① Compute:

$$z^{(i)} = \mathbf{w} \cdot \mathbf{x}^{(i)} + b$$

② Compute:

$$\hat{y}^{(i)} = \sigma(z^{(i)})$$

③ Compute loss.

④ Compute gradients.

⑤ Update weights and bias.

③ Stop when loss becomes small or maximum epochs reached.

Common Mistakes

- 1 Confusing z with $\hat{y}$.
- 2 Forgetting that $\hat{y}$ is a probability.
- 3 Writing log-odds incorrectly as $\log(p/p - 1)$.
- 4 Correct formula:

$$\log \left(\frac{p}{1 - p} \right)$$

- 5 Forgetting the bias update.
- 6 Forgetting to multiply by x_j in the weight gradient.
- 7 Updating in the direction of the gradient instead of the opposite direction.

Summary

- Logistic regression is used for classification.

- It starts with a linear score:

$$z = \mathbf{w} \cdot \mathbf{x} + b$$

- Sigmoid converts the score into probability:

$$\hat{y} = \sigma(z)$$

- Binary cross-entropy measures classification loss.
- Gradient descent updates parameters to reduce loss.
- Key gradient term:

$$\hat{y} - y$$

Logistic Regression = Linear Score + Sigmoid + Cross Entropy + Gradient Descent